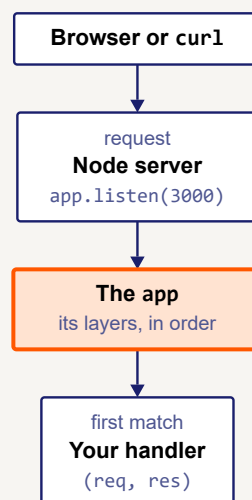


1. **Express and the app:** Express on top of Node: the app and app.listen · curl: sending requests from the terminal
2. **Routes and parameters:** Routes: method + path + handler · Route parameters: req.params · Query strings: req.query · Order of routes: the first match wins
3. **Requests and responses:** Reading the request: req · Answering: res · Exactly one response per request
4. **Middleware:** Middleware: (req, res, next) · express.json() and req.body · Middleware you will use: static, cors, logging · Guards: middleware that can stop a request · Order is configuration
5. **Structuring the app:** express.Router(): routes under a prefix · Controllers: thin routes, testable logic · app.js builds, server.js starts · The life of a request
6. **Errors and 404s:** The error handler: four parameters, registered last · Raising errors: throw and next(err) · Async handlers: Express 4 versus Express 5 · The 404 catch-all

Express and the app

Express on top of Node: the app and app.listen

Express is a small Node.js library: `express()` creates an **app** that holds your routes and middleware in order, and `app.listen(port)` starts the server that hands it every request.



Node receives the request; the app passes it through its layers to the handler that answers.

Routes

Method + path → a function: `app.get('/tasks', listTasks)` (see [Routes](#)).

Middleware

Functions every request passes through, in order: logging, parsing the body, checking a login (see [Middleware](#)).

Helpers on req and res

The request already parsed (`req.params`, `req.query`) and one-line answers (`res.status(201).json(task)`).

The app and app.listen

- `const app = express()` creates an empty app. Each `app.use`, `app.get` or `app.post` adds one layer, in the order the lines run.
- `app.listen(PORT, callback)` opens the port and hands every request to the app. The callback runs **once**, when the server is ready, not once per request.
- **The port** usually comes from the environment, `const PORT = process.env.PORT || 3000` (see [Environment variables](#)). The process then waits for requests until Ctrl+C.
- **Unopinionated:** Express chooses no folders, database or validation library; projects add their own conventions (see [A project layout](#)).

```
const express = require('express');
// an empty list of layers
const app = express();
```

```
// one route
app.get('/health', (req, res) => {
  res.json({ status: 'ok' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  // runs once, when ready
  console.log(`Listening on http://localhost:${PORT}`);
});
```

Example. Run it with `node --watch src/server.js`: the terminal prints `Listening on http://localhost:3000` once and stays busy. In a second terminal, `curl http://localhost:3000/health` prints `{"status":"ok"}`. Adding `GET /tasks/:id` is one more line, `app.get('/tasks/:id', getTask)`, with the `id` waiting in `req.params.id`; any unknown URL still gets a 404 from Express, without another `else`.

Common mistake. Thinking Express replaces Node. It is an npm package: `express()` returns an ordinary function that Node's http server calls for each request, so every error is a Node error and everything in Server-side JavaScript still applies. Starting it twice, for instance, gives Node's `EADDRINUSE` (see [A first server](#)).

curl: sending requests from the terminal

`curl` sends an HTTP request from the terminal and prints the response, so you can try any route, with any method, headers and body, without a browser or a front end.

<code>curl URL</code>	A GET request; prints the response body.
<code>-i</code>	Also prints the status line and the response headers.
<code>-X POST</code>	Chooses the method. Without it curl sends GET, or POST when there is a body.
<code>-H "Content-Type: application/json"</code>	Adds a request header; repeat it for more (<code>-H "Authorization: Bearer ..."</code>).
<code>-d '{"title":"Study"}'</code>	Sends a body. Without the Content-Type header curl labels it form data, and the server's JSON parser skips it (see express.json()).

```
curl -i http://localhost:3000/health
curl -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"title":"Study"}'
curl -X DELETE http://localhost:3000/tasks/1
```

Writing the JSON body in each terminal

Terminal	How
Git Bash, macOS, Linux	<code>-d '{"title":"Study"}'</code>
Windows PowerShell 5.1	Type <code>curl.exe</code> (plain <code>curl</code> there is another command) and escape the inner quotes: <code>-d '{"title":"Study"}'</code>
Command Prompt (cmd)	<code>-d "{\"title\":\"Study\"}"</code>

Example. `curl -i http://localhost:3000/health` prints the status line `HTTP/1.1 200 OK`, the headers (`Content-Type: application/json; charset=utf-8, X-Powered-By: Express...`), a blank line, and the body `{"status":"ok"}`. Without `-i` you see only the body.

Common mistake. Sending JSON with `-d` but no `-H "Content-Type: application/json"`. curl labels the body `application/x-www-form-urlencoded`, the server's JSON parser skips it, and the route sees an empty body: a 400 title is required although the JSON was right there.

Routes and parameters

Routes: method + path + handler

A **route** connects one HTTP method and one path pattern to a **handler function**: `app.get('/tasks', listTasks)` means "when a GET request for `/tasks` arrives, call `listTasks(req, res)`".

How a route matches

- **The whole path**, not its beginning: `/tasks` answers `/tasks`, not `/tasks/7`.
- **Forgiving by default**: a trailing slash is optional and letters are case-insensitive, so `/TASKS/` matches.
- **Never the query string**: `?done=true` plays no part.
- **No match**: Express answers 404 with an HTML page, Cannot GET `/x`. It sends no 405 when the path exists with another method: POST `/health` on an app with only GET `/health` is Cannot POST `/health`.

```
// GET    /tasks
app.get('/tasks', listTasks);
// POST   /tasks
app.post('/tasks', createTask);
// GET    /tasks/7, /tasks/abc ...
app.get('/tasks/:id', getTask);
// PATCH  /tasks/7
app.patch('/tasks/:id', updateTask);
// DELETE /tasks/7
app.delete('/tasks/:id', deleteTask);
```

Example. With the five routes above: GET `/tasks` → `listTasks`; GET `/Tasks/` → `listTasks` too (case and trailing slash ignored); PUT `/tasks/7` → no route has PUT, so Express answers 404 Cannot PUT `/tasks/7`; GET `/tasks/7/comments` → 404, because `/tasks/:id` has two segments and the URL has three.

Common mistake. Testing a POST route by typing its URL in the browser's address bar. The browser always sends GET, so you get Cannot GET `/tasks` and conclude the route is broken. Use [curl](#) with `-X POST`, or a fetch call with `method: 'POST'`.

Route parameters: req.params

A segment written `:name` in a route path is a **route parameter**: it matches any one segment of the URL and its value appears, as a **string**, in `req.params.name`.

- **Several parameters**: `/users/:userId/tasks/:taskId` fills one key each.
- **Always text**: the URL is text and Express does not guess types, so convert before comparing with numbers: `Number(req.params.id)`. Percent-escapes are decoded: `/tasks/a%20b` gives `'a b'`.
- **Any value fits**: `/tasks/:id` also matches `/tasks/stats`, with `id = 'stats'`. That is why route order matters (see [Order of routes](#)) and why a handler checks the value it receives.

```
app.get('/users/:userId/tasks/:taskId', (req, res) => {
  // GET /users/7/tasks/42
  console.log(req.params);
  // → { userId: '7', taskId: '42' }
  const taskId = Number(req.params.taskId);
  if (Number.isNaN(taskId)) {
    return res.status(400).json({ error: 'taskId must be a number' });
  }
  res.json({ userId: req.params.userId, taskId });
});
```

Example. An echo route GET `/echo/:msg` answers `{ "echo": req.params.msg }`. `curl http://localhost:3000/echo/hello` gives `{"echo":"hello"}`; `/echo/hello%20world` gives `{"echo":"hello world"}`; `/echo/hello/world` gives 404, because `:msg` takes exactly one segment.

Common mistake. Comparing the raw parameter with a number: `tasks.find((t) => t.id === req.params.id).7 === '7'` is `false` (strict equality compares types too), so the task is never found and every request answers 404. Convert first: `t.id === Number(req.params.id)`.

Query strings: `req.query`

The part of the URL after `?` holds optional `key=value` pairs; Express parses it into the object `req.query`, whose values are **strings** (or arrays of strings when a key repeats). It never takes part in route matching.

`?page=2` '2': every value is a string.
`?done=false` The string 'false', which is **truthy** in an `if`: compare as text, `=== 'true'`.
`?tag=css&tag=html` A repeated key becomes an array: `['css', 'html']`.
`?q=hello+world` `+` is a space.
`?filter[done]=true` An object in Express 4; Express 5 turns brackets off by default.

```
// What Express does with "?done=false&page=2&tag=css&tag=html&q=hello+world"
const params = new URLSearchParams('done=false&page=2&tag=css&tag=html&q=hello+world');

// a string
console.log(params.get('page'), typeof params.get('page'));
// repeated key
console.log(params.getAll('tag'));
// + is a space
console.log(params.get('q'));
// missing → null (undefined in req.query)
console.log(params.get('sort'));

const done = params.get('done');
if (done) console.log('"false" is a non-empty string, so this runs!');
console.log('compare as text:', done === 'true');
```

Example. A tasks API filters on the server: `GET /tasks?done=true`. A correct controller writes `const { done } = req.query; const list = done === undefined ? tasks : tasks.filter((t) => String(t.done) === done);`. `GET /tasks` returns everything, `?done=true` the finished tasks, `?done=false` the others.

Common mistake. Putting the query string in the route: `app.get('/tasks?done=true', ...)`. Express matches only the path, and `?` has a special meaning in Express 4 patterns, so this route never matches what you expect. Register `/tasks` and read `req.query.done` inside.

Order of routes: the first match wins

Express tries routes **in the order they were registered** and runs the first one whose method and path match. It never looks for the "most specific" route, so a pattern like `/tasks/:id` placed before `/tasks/stats` captures `stats` as an id.

```
// Wrong order: GET /tasks/stats → getTask with id 'stats' → 404
app.get('/tasks/:id', getTask);
// never reached
app.get('/tasks/stats', taskStats);

// Right order
// fixed path first
app.get('/tasks/stats', taskStats);
app.get('/tasks/:id', getTask);
```

Example. A tasks API needs a `GET /tasks/stats` endpoint. A developer adds it at the bottom of the file, under `app.get('/tasks/:id', getTask)`, and gets `{"error": "Task not found"}`. Nothing is wrong with the stats code: moving its line above the `:id` route fixes it.

Common mistake. "Express picks the best-matching route." It does not compare routes at all; it walks the list in order. The same goes for middleware (see [Order is configuration](#)): order in the file **is** the configuration.

Requests and responses

Reading the request: req

req is the request as an object: the method and URL, the route parameters, the parsed query string and body, and the headers. Express fills most of it before your handler runs.

The parts of req you will use (request: `PATCH /tasks/7?notify=true` with a JSON body)

Property	Value in the example	Filled by
<code>req.method</code>	<code>'PATCH'</code>	Node
<code>req.originalUrl</code>	<code>'/tasks/7?notify=true'</code>	Express: the full URL as sent
<code>req.path</code>	<code>'/tasks/7'</code>	Express: the path, without the query
<code>req.params</code>	<code>{ id: '7' }</code>	Express, from the route pattern <code>/tasks/:id</code>
<code>req.query</code>	<code>{ notify: 'true' }</code>	Express, from the query string
<code>req.body</code>	<code>{ done: true }</code>	The JSON parser, <code>express.json()</code> ; undefined without it (see express.json())
<code>req.get('Content-Type')</code>	<code>'application/json'</code>	Node (headers), read without caring about case
<code>req.user</code>	<code>{ id: 1, name: 'Ana' }</code>	Your own auth middleware, if any (see Guards)

- **Headers arrive lower-cased** in `req.headers` (`req.headers['content-type']`), whatever capitals the client used. `req.get('Content-Type')` ignores case, so prefer it.
- **Middleware writes on** `req` before your handler runs: the JSON parser fills `req.body`, an auth middleware adds `req.user` (see [Middleware](#)).
- **Inside a router** `req.path` loses the router's prefix, while `req.originalUrl` keeps the full URL (see [express.Router\(\)](#)).

Example. For that `PATCH /tasks/7?notify=true`: `Number(req.params.id)` is 7; `req.query.notify === 'true'` is true (compare as text); and `req.body.done` is true once the JSON parser has run, because JSON keeps booleans as booleans.

Common mistake. Reading `req.headers['Content-Type']` and getting undefined. Node lower-cases every header name, so the key is `'content-type'`. Use `req.get('Content-Type')`, which works with any capitalisation.

Answering: res

res builds the response: `res.status(code)` **sets the status**, `res.set(name, value)` **a header**, and **one sending method ends it:** `res.json(data)`, `res.send(text)`, `res.sendStatus(code)` **or** `res.end()`.

The res methods you will use most

Call	Sends?	Result
<code>res.status(404)</code>	No	Sets the status code; returns <code>res</code> for chaining
<code>res.set('Location', '/tasks/8')</code>	No	Sets a header (also <code>res.location(url)</code>)
<code>res.json({ id: 8 })</code>	Yes	JSON body with Content-Type: <code>application/json</code>
<code>res.send('<h1>Hi</h1>')</code>	Yes	Text body, Content-Type: <code>text/html</code> by default
<code>res.sendStatus(204)</code>	Yes	Only the status (and its reason as text, except 204)
<code>res.status(204).end()</code>	Yes	No body at all

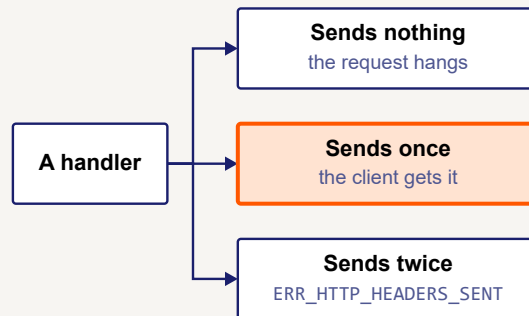
Example. A complete creation answer: `res.status(201).location(/tasks/${task.id}).json(task)`. On the wire: HTTP/1.1 201 Created, a Location: `/tasks/4` header, Content-Type: `application/json`; `charset=utf-8`, and the body `{"id":4,"title":"Study","done":false}`.

Common mistake. Writing `res.status(201)` and stopping there, expecting it to answer. It only sets a number: no sending method was called, so the request **hangs** until the client gives up. Every path through a handler must end with exactly one sending call (see [Exactly one response](#)).

Exactly one response per request

Every request must get one response: zero means the request hangs; two means Node throws Error

`[ERR_HTTP_HEADERS_SENT]: Cannot set headers after they are sent to the client.` **Write** `return res...` **whenever more code follows.**



Every path through a handler must send exactly once.

Where the two failures come from

- **Two responses:** an early answer without `return`. `res.status(400).json(...)` sends but does **not** stop the function, so the next lines run and try to send again. `return res.status(400).json(...)` sends and leaves in one step.
- **No response:** a path that sends nothing, such as an `if` with no `else` or a `catch` that only logs. Nothing times out on the server: the browser spins, and automated tests fail with a timeout.

```

// Bug: without return, a body without title sends TWO responses
function createTask(req, res) {
  const { title } = req.body;
  if (!title) res.status(400).json({ error: 'title is required' });
  // still runs!
  const task = { id: tasks.length + 1, title, done: false };
  tasks.push(task);
  res.status(201).json(task);
  // → Error [ERR_HTTP_HEADERS_SENT]
}

// Fixed: return stops the function after the 400
function createTask(req, res) {
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: 'title is required' });
  const task = { id: tasks.length + 1, title, done: false };
  tasks.push(task);
  res.status(201).json(task);
}
  
```

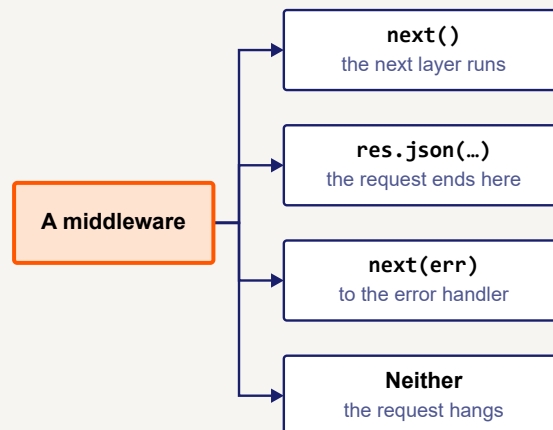
Example. With the buggy version, `POST /tasks` with `{}` returns `400` to the client, which looks right. But the function also pushed a task with `title: undefined` into the array, and the terminal shows `Error [ERR_HTTP_HEADERS_SENT]`. The next `GET /tasks` lists a task with no title: the bug corrupted data while the client saw a correct status.

Common mistake. Fixing the error message by wrapping the second `res.json` in `try/catch`. The real bug is that the code kept running after it answered; the fix is `return` (or `if/else`), so only one branch ever sends.

Middleware

Middleware: (req, res, next)

A **middleware** is a function (req, res, next) that Express runs for each request in registration order. It may change req or res, and then it must do one of two things: call next() to pass the request on, or send a response to end it. Doing neither leaves the request hanging.



A middleware ends in one of four ways; only one of them leaves the request stuck.

- app.use(fn) runs fn for every method and every path.
- app.use('/api', fn) runs it only for paths that **start with** /api, at a slash boundary: /api/tasks yes, /apiary no. That prefix matching is the big difference from routes, which match the whole path and one method.
- next(err) means "something failed": Express skips every ordinary layer up to the error handler (see [The error handler](#)).

```

// Logs, then passes the request on
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  // forget this and every request hangs
  next();
}

// Answers early, or passes on
function requireJson(req, res, next) {
  if (req.method === 'POST' && !req.is('application/json')) {
    // ends here
    return res.status(415).json({ error: 'Send JSON' });
  }
  next();
}

app.use(logger);
app.use(requireJson);
  
```

Example. Request GET /tasks on a typical tasks app (JSON parser, logger, /health, /echo/:msg, the tasks routes, 404, error handler): express.json() runs and calls next() (no body to read); logger prints GET /tasks and calls next(); GET /health and GET /echo/:msg do not match and are skipped; GET /tasks matches and listTasks answers 200. The layers after it (the 404 catch-all, the error handler) never run.

Common mistake. Writing a logger without next(). The terminal prints GET /health (so "the middleware works"), but the browser spins forever and curl never returns. Remember the rule: forget next() and every request hangs forever.

express.json() and req.body

`express.json()` is built-in middleware that reads the request body and, when the request has `Content-Type: application/json`, parses it into `req.body`. Without it `req.body` is undefined; with the wrong `Content-Type` it is `{}` (Express 4); with broken JSON it sends an error with status 400 to your error handler.

What a route finds in `req.body`

Situation	<code>req.body</code>	Why
No <code>express.json()</code> before the route	undefined	No parser ran
No body, or another <code>Content-Type</code>	<code>{}</code> in Express 4 (undefined in Express 5)	The parser skipped the body
Valid JSON	The object	Parsed
Broken JSON <code>{{title: x}}</code>	The route never runs: 400	The parser calls <code>next(err)</code> with <code>err.status = 400</code>
A body over 100 kB	The route never runs: 413	The default size limit

JavaScript + shell

```
// before the routes!
app.use(express.json());

app.post('/tasks', (req, res) => {
  console.log(req.body);
  // → { title: 'Buy milk' }
  res.status(201).json({ received: req.body });
});

// The client must say the body is JSON:
// curl -X POST http://localhost:3000/tasks \
//   -H "Content-Type: application/json" \
//   -d '{"title":"Buy milk"}'
```

Example. Run the `curl POST /tasks` command without the `-H "Content-Type: application/json"` line. `curl` then labels the body `application/x-www-form-urlencoded`, `express.json()` skips it, and `req.body` is `{}`: a controller that echoes the body answers `{"received":{}}`, and a validating one answers 400 title is required, although the JSON was right there.

Common mistake. Blaming the controller when `req.body` is undefined. That value means no body parser ran before the route: `express.json()` is missing or registered after it. `{}` means the parser ran but skipped the body: check the client's `Content-Type`.

Middleware you will use: static, cors, logging

A handful of middleware covers most projects: `express.json()` and `express.urlencoded()` (bodies), `express.static('public')` (files for the browser), `cors()` (calls from other origins) and a request logger (morgan or your own). The first three come with Express; `cors` and `morgan` are npm packages you install.

Common middleware

Middleware	Comes from	What it does	Register
<code>express.json()</code>	Express	JSON body → <code>req.body</code>	Early, before routes
<code>express.urlencoded({ extended: false })</code>	Express	HTML form body → <code>req.body</code>	Early, before routes
<code>express.static('public')</code>	Express	Sends files from a folder, else <code>next()</code>	Before the API routes
<code>cors()</code>	<code>npm install cors</code>	Adds CORS headers for browser calls from other origins	First, before routes
<code>morgan('dev')</code>	<code>npm install morgan</code>	One log line per request (method, URL, status, time)	Early
logger (yours)	Your file	Prints METHOD URL, then <code>next()</code>	Early

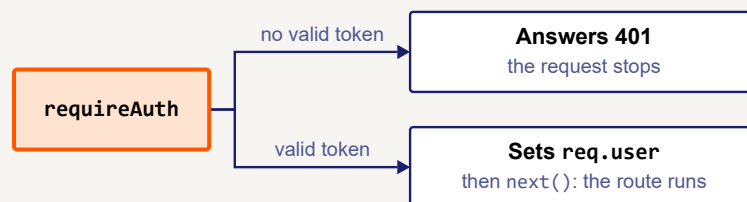
- `express.static` answers GET `/styles.css` with `public/styles.css` and `/` with `public/index.html`, and calls `next()` when no file matches, so the API routes below still work. The folder is resolved from where you ran node; `path.join(__dirname, '..', 'public')` is safer (see [Files and paths](#)).
- `cors()` lets a page from another origin read your responses. It protects nothing: browsers enforce CORS and curl ignores it (see [CORS: the server says who may read](#)).

Example. An `app.js` that serves a front end and an API starts with `app.use(cors()); app.use(express.json()); app.use(logger); app.use(express.static('public'))`; A browser asking for `/` gets `public/index.html` from `express.static`; the page's script calls `fetch('/api/tasks')`, which finds no file, falls through `express.static`, and reaches the tasks routes.

Common mistake. Error: Cannot find module 'cors': only `express.json`, `express.urlencoded` and `express.static` ship with Express. `cors` and `morgan` must be installed first (`npm install cors morgan`), which also adds them to dependencies in `package.json`.

Guards: middleware that can stop a request

A **guard** is middleware that checks something (usually who is calling) and either stops the request with an error response (401, 403) or adds what it learned to `req` (such as `req.user`) and calls `next()`.



A guard either answers and stops, or records who is calling and lets the request through.

```
function requireAuth(req, res, next) {
  const header = req.get('Authorization');
  // → "Bearer <token>"
  if (header !== 'Bearer demo-token') {
    return res.status(401).json({ error: 'Log in first' });
  }
  // a real app verifies a JWT here
  req.user = { id: 1, name: 'Ana' };
  next();
}

// public: above the guard
app.get('/api/health', health);
// guards everything below under /api
app.use('/api', requireAuth);
app.get('/api/me', (req, res) => res.json({ user: req.user }));
```

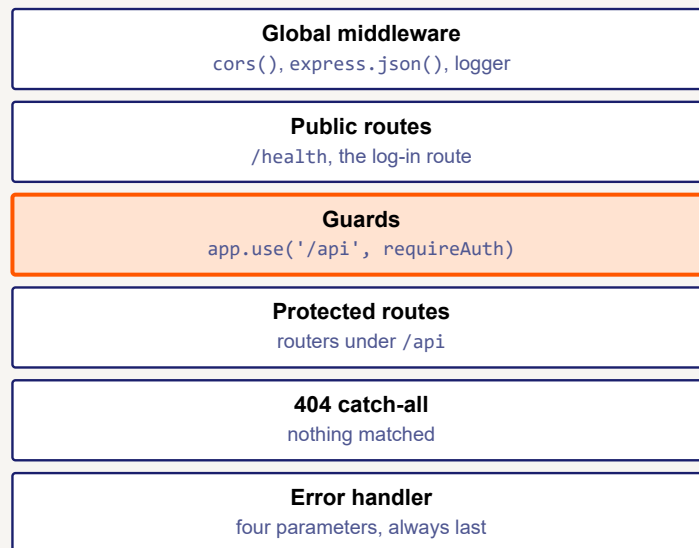
Example. `curl http://localhost:3000/api/me → 401 {"error":"Log in first"}`. `curl -H "Authorization: Bearer demo-token" http://localhost:3000/api/me → 200 {"user":{"id":1,"name":"Ana"}}`. `curl http://localhost:3000/api/health → 200` without a token, because that route is registered before the guard.

Common mistake. Registering the guard after the route it should protect. The route answers first, `req.user` is undefined, and `res.json({ user: undefined })` sends `{}` with status 200: no error anywhere, just an unprotected endpoint.

Order is configuration

Express runs layers in the order they were registered, so the order of `app.use` and route lines decides what each request meets: parsers and loggers first, guards before what they protect, routes, then the 404 catch-all, and the

error handler last.



Register in this order: each layer only affects the ones below it.

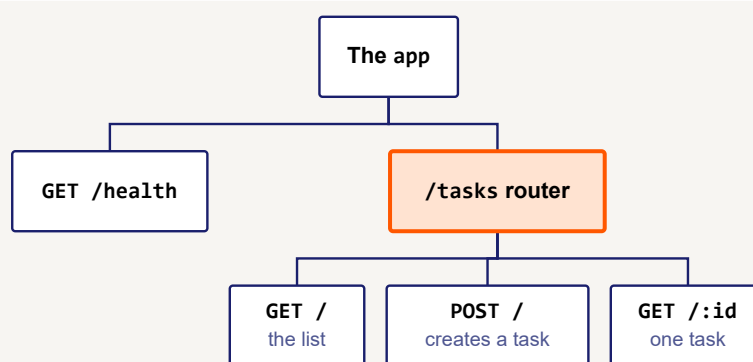
Example. In the "Protected API" preset of the pipeline simulator, GET /api/health is registered **above** `app.use('/api', requireAuth)`, so it stays public: health checks work without a token. GET /api/me is registered below the guard, so without Authorization: Bearer demo-token it answers 401.

Common mistake. Moving `app.use(express.json())` to the bottom of `app.js` "to keep the setup tidy". Every route above it now sees `req.body` as undefined, and POST /tasks crashes with Cannot destructure property 'title' of 'req.body'.

Structuring the app

`express.Router()`: routes under a prefix

`express.Router()` creates a mini-app that holds routes (and middleware) for one resource. `app.use('/tasks', tasksRouter)` mounts it: requests whose path starts with /tasks enter the router, and inside it the prefix is removed, so `router.get('/')` means GET /tasks.



Inside the router, paths are relative to its mount path.

`req.path` `'/7'` for GET /tasks/7: the path inside the router.

`req.baseUrl` `'/tasks'`: the mount path.

`req.originalUrl` `'/tasks/7'`: the full URL as sent.

- **No match inside:** the request leaves the router and continues with the next layer of the app, such as the 404 catch-all.
- **Parameters in the mount path** (`app.use('/users/:userId/tasks', router)`) reach the router's `req.params` only with `express.Router({ mergeParams: true })`.

```
// src/routes/tasks.js – thin: paths → controller functions
const express = require('express');
const { listTasks, createTask, getTask } = require('../controllers/tasksController');

const router = express.Router();
// GET /tasks
router.get('/', listTasks);
// POST /tasks
router.post('/', createTask);
// GET /tasks/:id
router.get('/:id', getTask);
module.exports = router;

// src/app.js
app.use('/tasks', require('./routes/tasks'));
```

Example. Say you move the API under /api. Only one line changes: `app.use('/api/tasks', tasksRouter)`. `GET /api/tasks/:id` now enters the router, which still sees /:id and still calls `getTask`; nothing in `routes/tasks.js` or the controller changes.

Common mistake. Repeating the prefix inside the router: `router.get('/tasks', listTasks)` while `app.js` mounts it at /tasks. The real path becomes /tasks/tasks, and `GET /tasks` answers 404. Inside a router, paths are relative to the mount path.

Controllers: thin routes, testable logic

A controller is the plain function that does a route's work (read the request, validate, fetch or change data, answer). Controllers live in `controllers/`, the route file only maps paths to them, so each part is short and the logic can be tested on its own.

```
// controllers/tasksController.js (no Express needed here)
const tasks = [{ id: 1, title: 'Write the API skeleton', done: true }];

function getTask(req, res) {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
}

// A fake res that remembers what the controller did
function fakeRes() {
  const res = { statusCode: 200, body: undefined };
  res.status = (code) => { res.statusCode = code; return res; };
  res.json = (data) => { res.body = data; return res; };
  return res;
}

const r1 = fakeRes();
getTask({ params: { id: '1' } }, r1);
console.log(r1.statusCode, JSON.stringify(r1.body));

const r2 = fakeRes();
getTask({ params: { id: '99' } }, r2);
console.log(r2.statusCode, JSON.stringify(r2.body));
```

Example. A small `routes/tasks.js` has three lines of real code: create a router, `router.get('/', listTasks)`, `router.post('/', createTask)`. When the project gets a database, the array inside `tasksController.js` is replaced by calls to a database model; the route file and `app.js` do not change at all.

Common mistake. Writing the logic inline in the route file "because it is only a few lines": `router.post('/', (req, res) => { ...30 lines... })`. The file grows unreadable and the logic cannot be tested on its own: keep handlers in `controllers/`. And a controller should never require the app or call `app.listen`, so that it stays a plain, testable function.

app.js builds, server.js starts

A well-organised Express project splits the program in two: `src/app.js` **builds** the Express app and exports it with `module.exports = app`; `src/server.js` **requires it and calls** `app.listen()`. Building and starting are separate jobs, so tests can use the app without opening a port.

- **One job per file:** `middleware/logger.js` exports one middleware, `routes/tasks.js` maps paths, `controllers/tasksController.js` does the work. The folder tree is in [project layout](#).
- **The file you run** is `server.js`: the start script runs `node --watch src/server.js`.

```
// src/app.js – builds and exports; never listens
const express = require('express');
const logger = require('./middleware/logger');
const tasksRouter = require('./routes/tasks');

const app = express();
app.use(express.json());
app.use(logger);
app.get('/health', (req, res) => res.json({ status: 'ok' }));
app.use('/tasks', tasksRouter);
// ... 404 catch-all and error handler last
module.exports = app;

// src/server.js – one job: start listening
const app = require('./app');
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Tasks API listening on http://localhost:${PORT}`));
```

Example. An automated test does `const request = require('supertest');` `const app = require('../src/app');` and then `await request(app).get('/tasks')`. `supertest` gives the app a temporary in-memory connection, so twenty tests can run in a row without port 3000 ever being opened, and without a server left running afterwards.

Common mistake. Forgetting `module.exports = app` at the end of `app.js`. Then `require('./app')` returns an empty object `{}`, and `server.js` crashes with `TypeError: app.listen is not a function`. The opposite mistake, calling `listen` inside `app.js`, makes every test open a real port, and a second test file fails with `EADDRINUSE`.

The life of a request

A request travels through one pipeline: Node receives it and hands it to the app; each middleware in order passes it on; the router picks the route; the controller answers. Any `next(err)` or thrown error jumps to the error handler; a request nobody answers reaches Express's own 404.

1. Node parses the request line and headers and calls the app.
2. Express adds its helpers (`req.query`, `req.get`, `res.json...`) and the X-Powered-By: Express header.
3. Each layer whose path matches runs in turn: middleware passes the request on, and the first matching route runs its controller.
4. No match: your 404 catch-all, or Express's `Cannot GET /x`.
5. On `next(err)` or a thrown error: everything else is skipped until the four-parameter error handler.

A response From a controller, a guard, `express.static`, your 404 or your error handler, or Express's own final handler.

A hang Some layer neither answered nor called `next()`.

A crash An error nobody caught ends the whole process, such as a rejected promise in an Express 4 `async` handler (see [Async handlers](#)).

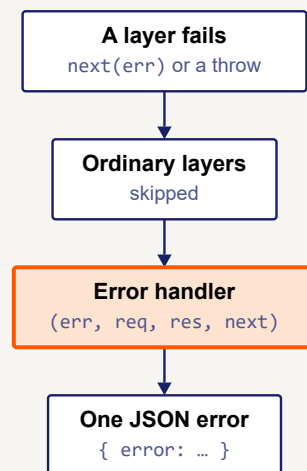
Example. `POST /tasks` with a broken JSON body: `express.json()` fails to parse it and calls `next(err)` with status 400 (step 3); the logger, the routes and the 404 catch-all are all skipped, because an error is pending (step 5); the error handler answers `400 {"error":"Expected property name or '}' in JSON at position 1 ..."}.` The controller never ran, and the logger never printed a line.

Common mistake. Assuming every middleware runs for every request. A layer only runs if the request **reaches** it: an earlier response ends the journey, an error makes Express skip ordinary layers, and `app.use('/api', ...)` never sees `/health`.

Errors and 404s

The error handler: four parameters, registered last

An **error-handling middleware** is a function with **four parameters**, (`err`, `req`, `res`, `next`). Express recognises it only by that count, skips it during normal requests, and sends it every error passed with `next(err)` or thrown in a handler. Register one, after every route.



An error skips straight to the next four-parameter function.

- **Recognised by its parameter count** (`fn.length === 4`): with three it is ordinary middleware, even if the first one is named `err`. Keep `next` even if you never call it.
- **One shape for every failure:** log the error, then `res.status(err.status || 500).json({ error: err.message })`. Errors you create can carry their status: `err.status = 404`.
- **Without one**, Express's final handler sends a **500 HTML** page with the stack trace (only Internal Server Error when `NODE_ENV=production`): the wrong format for an API, and in development it leaks file paths.

```
// How Express tells the two kinds apart: it counts the parameters
const errorHandler = (err, req, res, next) => {};
const looksLikeOne = (err, req, res) => {};

console.log('4 parameters ->', errorHandler.length, '(error handler)');
console.log('3 parameters ->', looksLikeOne.length, '(ordinary middleware!)');

// Express 4, simplified (Layer.handle_error / handle_request):
function isErrorHandler(fn) { return fn.length === 4; }
console.log(isErrorHandler(errorHandler), isErrorHandler(looksLikeOne));
```

Example. A test route `GET /boom` calls `next(new Error('Boom!'))`. With `app.use((err, req, res, next) => { console.error(err.message); res.status(err.status || 500).json({ error: err.message }); })` at the bottom, `curl` prints `{"error":"Boom!"}` with status 500, the terminal prints `Boom!`, and the server keeps running.

Common mistake. Deleting the "unused" `next` parameter from the error handler to please a linter. With three parameters it is no longer an error handler: errors skip it, the client gets Express's HTML error page, and on normal requests that reach it Express calls it as (`req`, `res`, `next`), so your `res.status` is really `next.status` and it throws `TypeError: res.status is not a function`.

Raising errors: throw and next(err)

There are two ways to send a failure to the error handler: call `next(err)` (works everywhere), or throw inside a handler while it is still running synchronously (Express catches it). From then on Express skips ordinary layers until a four-parameter error handler.

- return `next(err)` when you decide something failed, with a status for the handler:
`err.status = 404`. The return stops the function, for the same reason as `return res.....`
- **Pass an Error object**, never a string: the handler reads `err.message` and `err.status`.
- **Not caught by Express 4**: an error thrown *later*, after an `await` or inside a callback, once your handler has returned (see [Async handlers](#)).

```
// Both reach the error handler
app.get('/boom', (req, res, next) => {
  // explicit
  next(new Error('Boom!'));
});
app.get('/boom2', (req, res) => {
  // caught by Express (synchronous)
  throw new Error('Boom!');
});

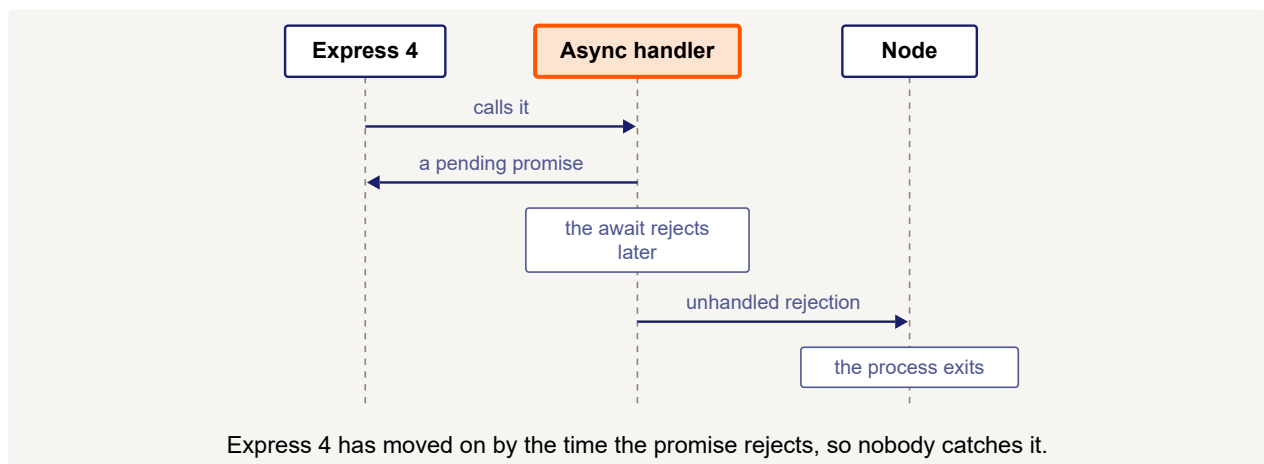
// An error with a status, for the central handler
function getTask(req, res, next) {
  const task = tasks.find((t) => t.id === Number(req.params.id));
  if (!task) {
    const err = new Error('Task not found');
    err.status = 404;
    return next(err);
  }
  res.json(task);
}
```

Example. In a tasks app without `express.json()`, `POST /tasks` makes `createTask` run `const { title } = req.body` with `req.body` undefined. JavaScript throws `TypeError: Cannot destructure property 'title' of 'req.body' as it is undefined.`; Express catches it, skips the 404 catch-all, and the error handler answers 500 with that message. The server survives.

Common mistake. Calling `next('Task not found')` with a plain string. Express treats it as an error, but the handler's `err.message` and `err.status` are undefined, so the client gets 500 and `{}`. Always pass an Error object (with a status when it is not a 500).

Async handlers: Express 4 versus Express 5

In **Express 4**, an async handler whose promise rejects never reaches your error handler: on Node 15+ the unhandled rejection **crashes the server**. Catch it yourself with `try/catch` and `next(err)`. **Express 5** forwards rejected promises to the error handler automatically.



The same async handler in both versions

	Express 4	Express 5
A rejected promise	Not caught: the process crashes	Forwarded to the error handler
What you write	try { ... await ... } catch (err) { next(err); }, or a wrapper	Nothing extra (the try/catch still works)

```
// Express 4: a rejection here crashes the process
app.get('/stats', async (req, res) => {
  // rejects: database offline
  const stats = await db.getStats();
  res.json(stats);
});

// Works in Express 4 and 5
app.get('/stats', async (req, res, next) => {
  try {
    const stats = await db.getStats();
    res.json(stats);
  } catch (err) {
    next(err);
    // → error handler → 500 JSON
  }
});

// Optional helper once the pattern is familiar
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
router.get('/', asyncHandler(listTasks));
```

Example. In the pipeline simulator, preset "Bug: async route without try/catch": GET /stats with Express 4 ends in a crash and curl prints Empty reply from server. Switch the version to Express 5 and the same code answers 500 {"error":"Database is offline"}; or stay on 4 and choose the try/catch + next(err) code.

Common mistake. Adding try/catch but only logging: catch (err) { console.error(err); }. The crash is gone, but now nothing answers the request, so it hangs. The catch must end the request too: next(err), or a response of its own.

The 404 catch-all

When no layer answers, Express sends its own 404 HTML page Cannot GET /x. **An API adds a catch-all middleware after every route**, app.use((req, res) => res.status(404).json({ ... })), **so unknown URLs get JSON too.**

No such route GET /tasks: the catch-all's job.

No such resource GET /tasks/999: the route exists, so the controller looks for the task and answers 404 Task not found itself.

```
// ... every route and router above ...

app.use((req, res) => {
  // → 404 catch-all
  res.status(404).json({ error: `Not found: ${req.method} ${req.originalUrl}` });
});

// error handler, last
app.use((err, req, res, next) => {
  console.error(err.message);
  res.status(err.status || 500).json({ error: err.message });
});
```

Example. Without the catch-all, curl -i http://localhost:3000/nope shows HTTP/1.1 404 Not Found, Content-Type: text/html and a small HTML page with <pre>Cannot GET /nope</pre>. With it: 404, Content-Type: application/json and {"error":"Not found: GET /nope"}, which a fetch call can read with response.json().

Common mistake. Registering the catch-all near the top of `app.js`, next to the other `app.use` lines. It matches every request, so every route below it is unreachable and the whole API answers 404. Its place is after the last route.